

Pashov Audit Group

# TokenLogic Security Review

October 28th 2026

# Contents

1. About Pashov Audit Group 3
2. Disclaimer 3
3. Risk Classification 3
4. Methodology 4
5. Executive Summary 4
6. Findings 4
Info Severity 4
[I-01] Revoked signers cannot cancel pending signed swaps, which become valid again when access reopens 4
[I-02] Revoking a participant needs a governance vote plus a 1-day delay and can be front-run 5
[I-03] Setting the GSM's reserve limit to 0 leaves the last participant unable to unwind 6
[I-04] A role-holding contract opens the GSM to anyone, and `receiver` is never checked 7
[I-05] `CONFIGURATOR_ROLE` can open access to everyone but cannot grant it to one address 7

# 1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Telegram @pashovkrum.

# 2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

# 3. Risk Classification

| Severity | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| Likelihood: High | Critical | High | Medium |
| Likelihood: Medium | High | Medium | Low |
| Likelihood: Low | Medium | Low | Low |

| | |
|---|---|
| Critical | Loss or compromise of critical assets, or protocol-wide impact. |
| High | Significant material loss of assets, or serious harm to a group of users. |
| Medium | Moderate material loss, or moderate harm, often under conditional assumptions. |
| Low | Minor impact, limited scope, or requires unlikely conditions to trigger. |
| Info | No direct security impact - best-practice, code-quality or hardening suggestion. |

### IMPACT

**High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users

**Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users

**Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

### LIKELIHOOD

**High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

**Medium** - only a conditionally incentivized attack vector, but still relatively likely

**Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

## 4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

## 5. Executive Summary

A time-boxed security review of TokenLogic was conducted by the Pashov Audit Group team. This report documents 5 issues across the reviewed codebase, reported by said017, shaka, l33.

**SCOPE**

https://github.com/aave-dao/gho-origin/pull/49

## 6. Findings

| ID | TITLE | SEVERITY | STATUS |
|---|---|---|---|
| I-01 | Revoked signers cannot cancel pending signed swaps, which become valid again when access reopens | Info | Acknowledged |
| I-02 | Revoking a participant needs a governance vote plus a 1-day delay and can be front-run | Info | Fixed |
| I-03 | Setting the GSM's reserve limit to 0 leaves the last participant unable to unwind | Info | Acknowledged |
| I-04 | A role-holding contract opens the GSM to anyone, and `receiver` is never checked | Info | Acknowledged |
| I-05 | `CONFIGURATOR_ROLE` can open access to everyone but cannot grant it to one address | Info | Fixed |

## Info Severity

### [I-01] Revoked signers cannot cancel pending signed swaps, which become valid again when access reopens

Info Acknowledged

**DESCRIPTION**

A nonce is only consumed by executing a swap. There is no function to invalidate one:

```
function sellAssetWithSig(
address originator,
...
) external notFrozen notSeized returns (uint256, uint256) {
...
>>>         nonces[originator]++,
...
return _sellAsset(originator, maxAmount, receiver);
}
```

Once a signer loses the role, every swap they originate reverts in `_checkRole`, which also undoes the nonce increment:

```
function _sellAsset(...) internal override returns (uint256, uint256) {
if (!_isSellAccessOpen) {
>>>     _checkRole(SELL_ASSET_ROLE, originator);
}
return super._sellAsset(originator, maxAmount, receiver);
}
```

So the signer cannot cancel an outstanding signature. If access is opened later or the role is granted again, any relayer can execute the old signature before its deadline.

Buy and sell signatures share one nonce, so one stale signature for a revoked side also blocks every later signature from the same signer.

Consider to add a role-free `invalidateNonce()` that increments `nonces[msg.sender]`.

## [I-02] Revoking a participant needs a governance vote plus a 1-day delay and can be front-run

Info Fixed

**DESCRIPTION**

Only governance can restrict access:

```
// BUY_ASSET_ROLE / SELL_ASSET_ROLE are administered by DEFAULT_ADMIN_ROLE
>>> function setBuyAccessOpen(bool open) external onlyRole(CONFIGURATOR_ROLE) {
>>> function setSellAccessOpen(bool open) external onlyRole(CONFIGURATOR_ROLE) {
```

In the ARFC for the Avalanche USAT GSM, both roles stay with governance (Executor Lvl 1 `0x3C06dce358add17aAf230f2234bCCC4afd50d090`). The Avalanche PayloadsController (`0x1140CB7CAfAcC745771C2Ea31e7B5C653c5d0B80`) adds an 86400-second delay after the vote. No guardian can revoke a role or close access.

A participant being off-boarded can see the payload and keep swapping until it executes. The only faster option is `setSwapFreeze`, which stops all participants.

Consider add a guardian role that can only revoke `BUY_ASSET_ROLE` / `SELL_ASSET_ROLE` and close access, without being able to grant or open.

## [I-03] Setting the GSM's reserve limit to 0 leaves the last participant unable to unwind

Info Acknowledged

**DESCRIPTION**

This is existing `Gsm4626` and `GhoReserve` code. It matters for the Avalanche USAT GSM proposed in the ARFC: participants share liquidity, the GSM holds waUSAT (the yield-bearing Aave V4 receipt token for USAT), and the reserve limit is set separately by the GHO Stewards Safe ( `0x8513e6F37dBc52De87b166980Fa3F50639694B60` ).

Yield is only booked while the reserve has headroom:

```
function _cumulateYieldInGho() internal {
(uint256 ghoLimit, uint256 ghoUsed) = _getUsage();
>>>  uint256 ghoAvailableToMint = ghoLimit > ghoUsed ? ghoLimit - ghoUsed : 0;
(uint256 ghoExcess, ) = _getCurrentBacking(ghoUsed);
>>>  if (ghoExcess > 0 && ghoAvailableToMint > 0) {
...
IGhoReserve(_ghoReserve).use(ghoExcess);
}
}
```

But every buy repays the reserve at the current share price, and `restore` underflows if that exceeds `used` :

```
>>>  IGhoReserve(_ghoReserve).restore(grossAmount);
```

```
function restore(uint256 amount) external {
require(amount > 0, 'INVALID_AMOUNT');
>>>  _ghoUsage[msg.sender].used -= amount.toUint128();
```

With the limit at 0, the unbooked yield makes total repayments exceed `used` , so the last buys revert. Example: two participants sell 1M shares each, the share price rises 2%, and the limit is set to 0. The first participant buys back 1M shares. The second can only buy back 960,784; the remaining 39,216 are stuck until the limit is raised. The treasury's yield is stuck too, and `removeEntity` can never pass.

Consider to :

- Wind down with `updateExposureCap(0)` instead of `setLimit(0)` , and call `distributeFeesToTreasury()` before lowering the limit.
- Book yield before repaying in `_buyAsset` , or cap `restore` at `used` and send the surplus to fees.

## [I-04] A role-holding contract opens the GSM to anyone, and `receiver` is never checked

Info Acknowledged

**DESCRIPTION**

Only the originator is checked:

```
if (!_isSellAccessOpen) {
>>>     _checkRole(SELL_ASSET_ROLE, originator);
}
return super._sellAsset(originator, maxAmount, receiver);
```

If a contract with a public swap function (router, aggregator, smart account) gets a role, anyone can swap through it. Output can go to any `receiver`. The ARFC for the Avalanche USAT GSM accepts unchecked recipients and relayers.

Consider to only grant roles to EOAs or contracts that cannot swap for third parties.

## [I-05] `CONFIGURATOR_ROLE` can open access to everyone but cannot grant it to one address

Info Fixed

**DESCRIPTION**

```
>>> function setBuyAccessOpen(bool open) external onlyRole(CONFIGURATOR_ROLE) {
>>> function setSellAccessOpen(bool open) external onlyRole(CONFIGURATOR_ROLE) {
```

The swap roles are administered by `DEFAULT_ADMIN_ROLE`, yet a configurator can open the GSM to every address. On the existing Ethereum GSMs the GHO GSM Steward holds `CONFIGURATOR_ROLE`, and the ARFC for the Avalanche USAT GSM gives it the same permissions there. The current steward cannot call these setters. A future configurator with a generic call path could.

Consider to gate both setters with `DEFAULT_ADMIN_ROLE` or a dedicated role.